

Coding & Solution

Date	02 July 2026
Team ID	
Project Name	Personalized Networking Assistant
Maximum Marks	5 Marks

Coding & Solution Report

This section evaluates the quality and completeness of your implemented code and the overall solution.

Solution Summary

Field	Details
Repository Link / URL	https://github.com/Srikar-jayanthi/Personalized-Networking-Assistant.git
Programming Language(s)	Python 3.11+
Framework(s) Used	FastAPI (Backend), Streamlit (Frontend)
Key Features Implemented	• Event theme extraction using zero-shot DistilBERT • Tailored prompt generator utilizing GPT-2 • Wikipedia real-time fact checker API integration • Local JSON session logs and thumbs up/down voting feedback logs
Pending / Incomplete Features	None. Epics 1 & 2 fully implemented and validated.
Setup / Run Instructions	1. Install requirements: <code>pip install -r requirements.txt</code> 2. Start backend server: <code>uvicorn app.main:app --port 8000</code> 3. Start frontend Streamlit UI: <code>streamlit run frontend/streamlit_app.py</code> 4. Launch browser views.

Code Quality Checklist

S.No	Criteria	Status (Yes/No)
1	Code is modular and organized into functions / classes.	Yes
2	Meaningful variable and function names are used.	Yes
3	Code includes comments / documentation where necessary.	Yes
4	Error handling is implemented for critical operations.	Yes
5	The application runs without critical errors.	Yes
6	Code is committed to a version control repository.	Yes

Core Backend Framework Architecture:

The backend REST structure is built on FastAPI. The core routing layer parses JSON inputs and initiates backend model pipelines. PyTest unit checks verify that zero-shot classification maps descriptions correctly, and mock requests tests cover all Wikipedia fact checking paths. All code has been formatted, audited, and committed to git, leaving the local template files clean of untracked cache folders.